DAY 06 · THEORY HANDOUT

Logistic Regression: Choose a Class

Turn inputs into a probability, choose a class, and measure the kinds of mistakes.

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

One input and one class label

The question is whether a student will pass. The input is study hours.

Study hours: feature x	Exam result: target y
1	0: did not pass
2	0: did not pass
3	1: passed
4	1: passed

Example records. The labels are known during training.

Binary classification chooses between two classes. Here, 0 means did not pass and 1 means passed.

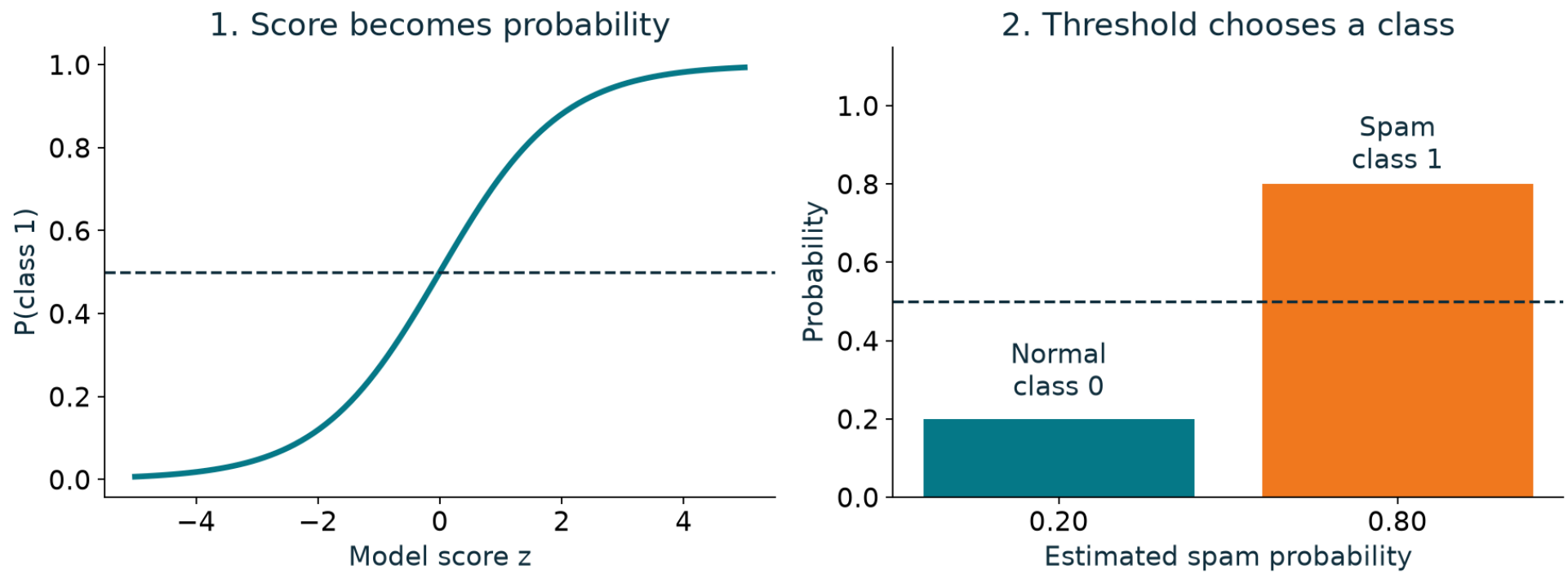
The known labels guide training. Logistic Regression learns values that turn input into an estimated probability of class 1.

Despite the word regression in its name, this model predicts a class after converting its probability with a threshold.

These are example records. Study time alone cannot guarantee whether a student passes.

From a score to a class

A probability and a class label are two different outputs.



With a 0.5 threshold, a spam probability of **0.20 gives normal**; **0.80 gives spam**.

> Changing a threshold can change the label without changing the probability.

Step 1: calculate a score

A trial model has weight 1 and intercept -3. Score z = 1 x study hours - 3.

Study hours	Score z = 1 x hours - 3	Score z
1	1 x 1 - 3	-2
2	1 x 2 - 3	-1
3	1 x 3 - 3	0
4	1 x 4 - 3	1

A **weight** says how an input changes the score. The **intercept** shifts the score up or down.

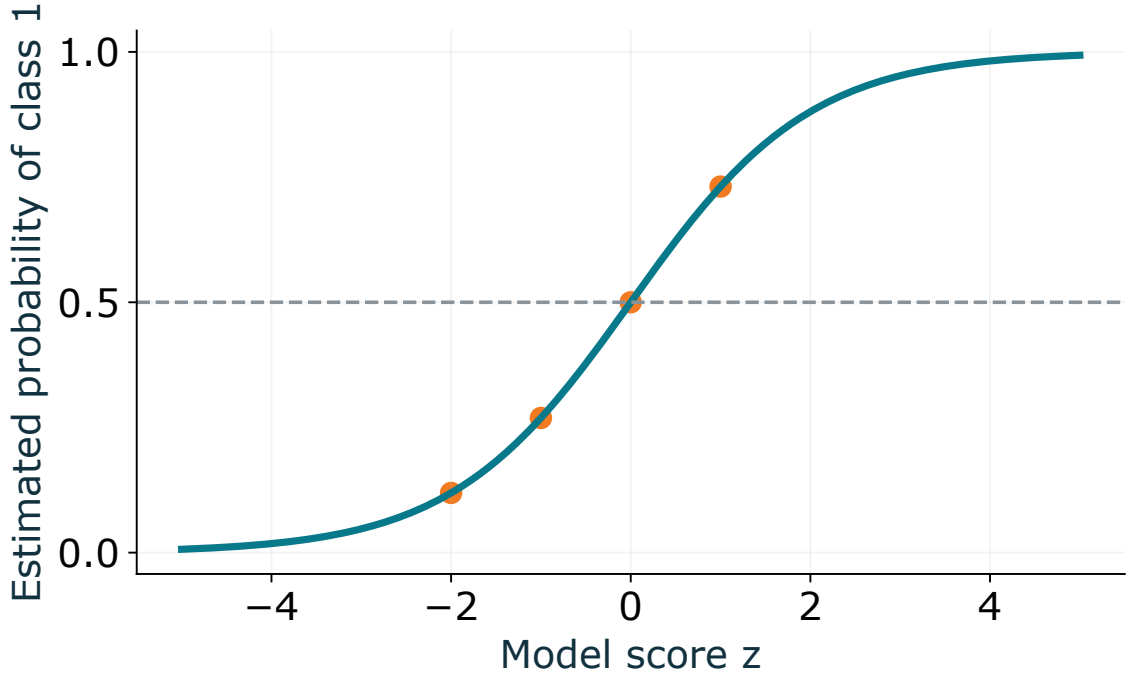
For 1 study hour: z = 1 x 1 - 3 = -2. For 4 study hours: z = 1 x 4 - 3 = 1.

This score can be negative or positive. It is not yet a probability. A probability must stay between 0 and 1.

Training changes the weight and intercept. The next step turns each score into a probability.

Step 2: apply sigmoid

Sigmoid is a function that converts the score into a probability between 0 and 1.



The S-shaped curve gives **sigmoid(-2) = about 0.12**, sigmoid(-1) = about 0.27, sigmoid(0) = 0.50, and sigmoid(1) = about 0.73.

A higher score gives a higher estimated probability of class 1. A score of 0 gives equal estimated chances for the two classes.

The model uses this fixed curve. Training changes the scores by changing the weight and intercept.

A probability of 0.73 means an estimated 73% chance of class 1. It is not a guarantee.

Step 3: turn probability into a class

A threshold is the cut point used to make a class decision.

Hours	P(pass)	Predicted class	Actual class
1	0.12	0	0
2	0.27	0	0
3	0.50	0	1
4	0.73	1	1

Rule used here: class 1 if probability > 0.5; otherwise class 0.

With the rule shown here, a probability above **0.5** gives class 1. A probability of 0.5 or below gives class 0.

The 3-hour record is wrongly called class 0 even though its actual label is class 1. The candidate has one wrong class prediction.

The threshold can be changed for a task. A lower threshold usually gives more positive calls; a higher threshold usually gives fewer.

Exactly at the threshold, a tie rule is needed. This example assigns the tie to class 0.

Step 4: measure probability mistakes

Training checks the estimated probability against the known label, not just the final class.

Actual class	P(class 1)	Log loss	Meaning
1	0.90	0.11	Likely correct
1	0.10	2.30	Confident but wrong
0	0.10	0.11	Likely correct
0	0.90	2.30	Confident but wrong

Lower loss is better. The numbers are rounded. No manual logarithm calculation is needed to read them.

Log loss measures probability mistakes. A correct, confident probability gets a small loss.

If the actual label is 1 but the model gives only a 0.10 probability of class 1, the loss is large. The model was confident in the wrong direction.

The average training loss helps judge candidate weights. Sigmoid converts scores; log loss judges probabilities. They have different jobs.

> A residual or squared error from Linear Regression is not the loss used in this Logistic Regression example.

Step 5: improve the weights

Compare the same four known labels before and after a change in model values.

Hours	Actual class	Before: P(pass)	After: P(pass)
1	0	0.119	0.047
2	0	0.269	0.269
3	1	0.500	0.731
4	1	0.731	0.953

Before: weight 1, intercept -3; mean log loss 0.362
After: weight 2, intercept -5; mean log loss 0.181

The new candidate uses **weight 2** and **intercept -5**. Its probabilities put more chance on the known classes.

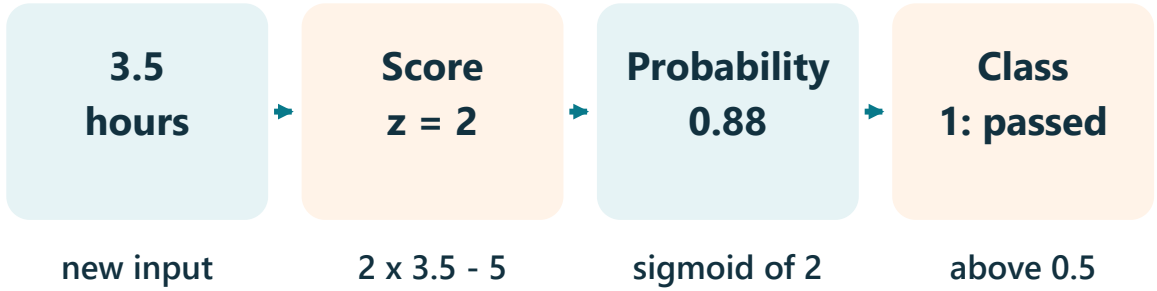
The mean log loss falls from **0.362** to **0.181**. The new candidate also calls all four training classes correctly.

A fitting method keeps changing values to improve its training objective. **Regularization** adds a cost for very large weights, to help avoid fitting only training details.

> Four correct training calls do not prove that predictions on new students will be correct.

Step 6: predict a new class

Use the improved candidate to predict a result for 3.5 study hours.



Score: 2 x 3.5 - 5 = 2. **Sigmoid:** the estimated probability of passing is about 0.88.

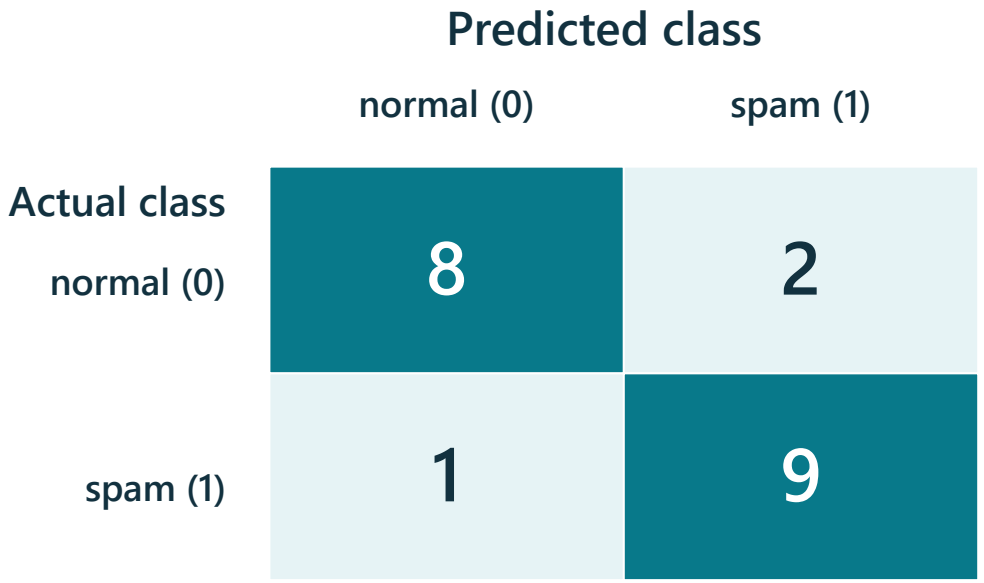
Threshold: 0.88 is above 0.5, so the predicted class is 1: passed.

The weight, intercept, sigmoid function, and threshold stay fixed while making this prediction. The real result can be checked when it is known.

Keep some labeled records out of training so the model can be checked on unseen examples.

A confusion matrix counts mistakes

A spam filter predicts 20 messages. Spam is the positive class, written 1.



True Positive (TP) = 9: spam correctly flagged. **True Negative (TN) = 8:** normal email correctly left alone.

False Positive (FP) = 2: normal email wrongly flagged. **False Negative (FN) = 1:** spam that was missed.

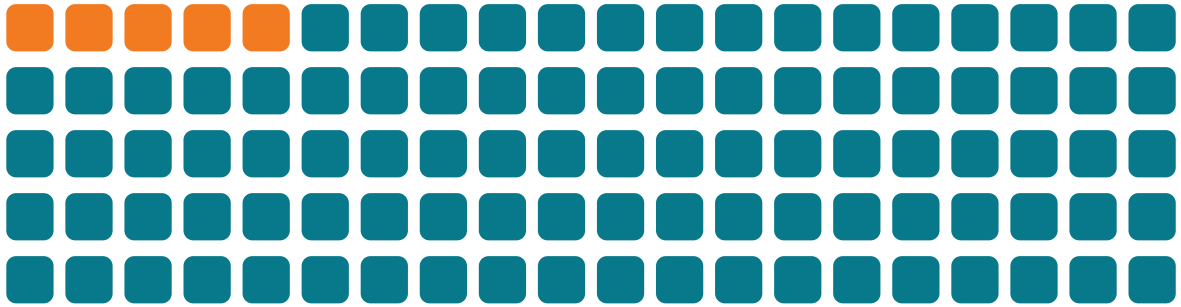
Rows show actual classes. Columns show predicted classes. The diagonal holds correct predictions.

> Positive means the class being counted as 1. It does not mean good or safe.

Accuracy: how many were right?

Accuracy counts correct predictions across all classes.

100 messages: 95 normal, 5 spam



Always say normal: 95% accuracy, 0% spam recall

In the 20-message matrix, 8 normal emails and 9 spam messages were right. **Accuracy = 17 / 20 = 85%.**

Use accuracy when class sizes and mistake costs are fairly similar. Also read the confusion matrix.

With 95 normal messages and only 5 spam messages, always saying normal gives 95% accuracy but finds no spam. High accuracy can hide this failure.

Class imbalance means one class has many more records than another.

Precision and recall: which mistake matters?

The same model can have two different scores because each metric counts a different group.

Metric	Question	Same 20-message example
Precision	Were positive calls right?	9 real spam / 11 spam calls = 81.8%
Recall	Were real positives found?	9 found / 10 real spam = 90%

Precision checks the predicted positive group.
Recall checks the actual positive group.

Use precision when wrong positive calls are costly. A spam filter should not hide important normal email by mistake.

Use recall when missing positive cases is costly. If many real spam messages reach the inbox, check how much spam was missed.

Another recall example is finding broken items before packing. Out of 10 broken items, finding 9 gives recall = 90%. The positive class there is broken.

> Precision asks "Were the flags right?" Recall asks "Did the model find the real cases?"

F1 and choosing a metric

Choose the metric by the cost of a wrong call or a missed case.

Measure	What matters	Example use
Accuracy	All correct calls	Similar class sizes and mistake costs
Precision	Wrong positive calls	Avoid hiding normal email as spam
Recall	Missed positive cases	Find more of the actual spam
F1	Precision and recall together	Both wrong flags and missed cases matter

For the same matrix: $F1 = 2 \times 9 / (2 \times 9 + 2 + 1) = 0.857$

F1 score combines precision and recall. It is their harmonic mean: an average that stays low when either one is low.

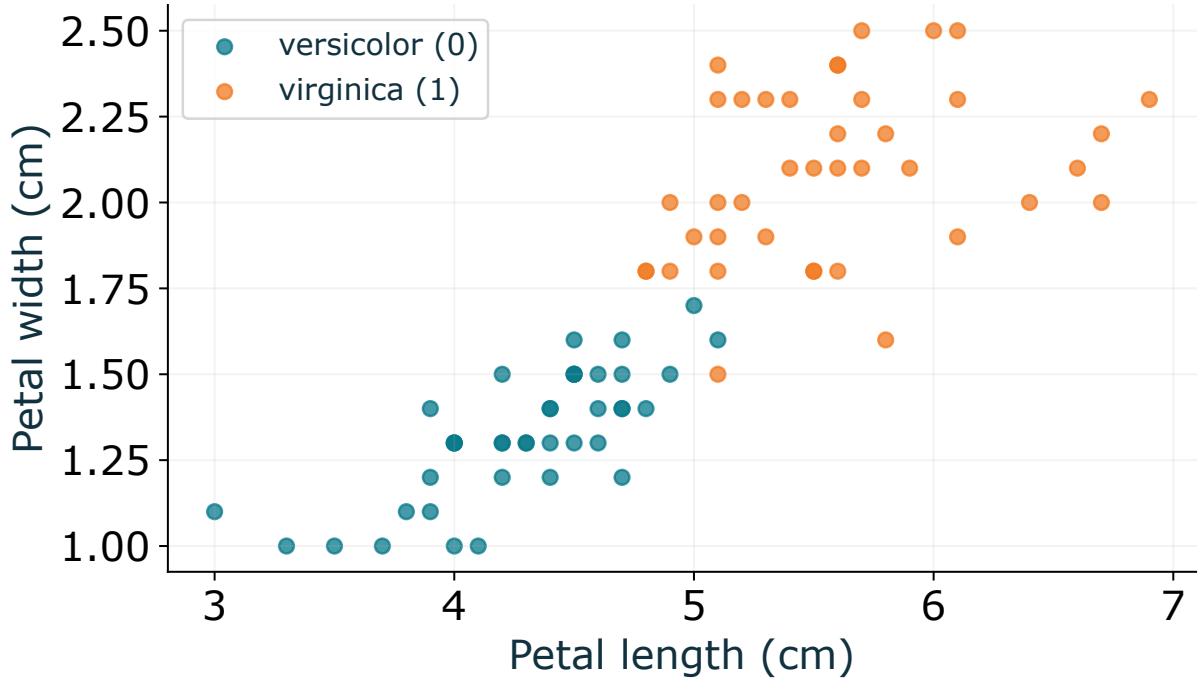
Use F1 when both wrong positive calls and missed positive cases matter. F1 does not count true negatives.

A single score does not choose the best model for every task. Start with the confusion matrix and decide which mistake matters most.

For the 20-message example, precision is 81.8%, recall is 90%, and F1 is about 85.7%.

An example with flower measurements

The iris data contains measured petals and known species names.



This example selects 100 flowers from two species. The **features** are petal length and width. A petal is a colored part of a flower.

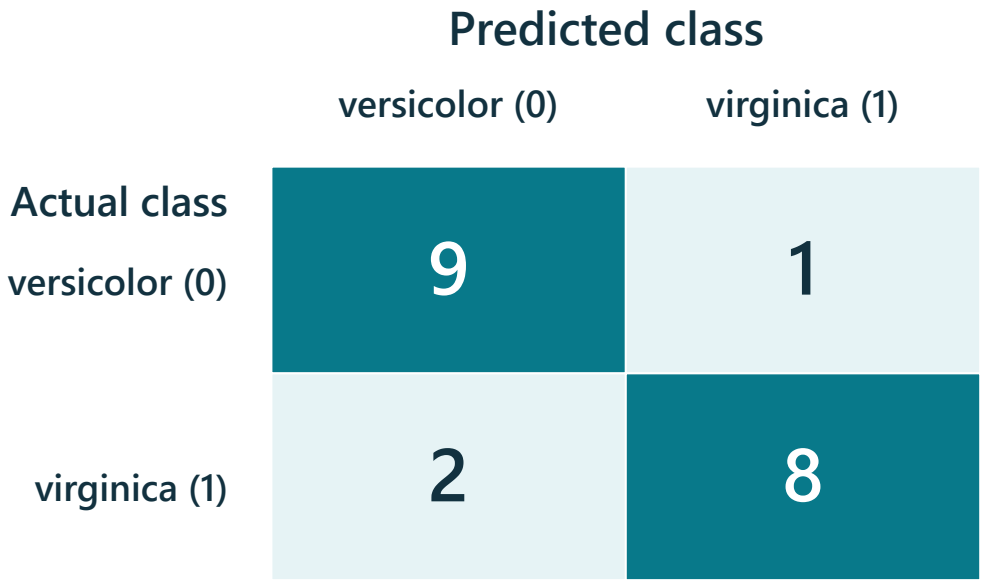
The **target** is species: 0 is versicolor and 1 is virginica. The split has 80 training flowers and 20 held-out test flowers.

The feature values are scaled using training data so their ranges are comparable. The same learned scale is then applied to new input.

The classes overlap in some measurements. A model can still confuse one species with the other.

Read the held-out predictions

Logistic Regression predicts 20 flowers that were not used to train it.



Accuracy = 85%: 17 of 20 flowers were correct. There are 9 correct class-0 calls and 8 correct class-1 calls.

Precision = 88.9%: 8 of 9 flowers called virginica really were virginica. **Recall = 80%:** 8 of 10 real virginica flowers were found.

F1 = 84.2%. One versicolor flower was wrongly called virginica, and two virginica flowers were missed. Class 1 is virginica for these metrics.

Only 20 flowers were tested. One changed prediction moves accuracy by 5 percentage points.

DAY 06 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie